SCHEDULERS

ALEXEY TUMANOV

School of Computer Science
faculty.cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

MOMENT OF INSPIRATION

Georgia Tech

Operate on

a different

axiomatic base

ADMINISTRATIVIA

- **Lab 2 – Due THIS Friday, 2/27!**
  - **3 days from today!**
  - **If you haven't yet submitted, do so ASAP. It will likely take many submissions to pass all tests. (20 submissions)**
  - **Last supervised lab TODAY before the deadline**
    - **Extended Office Hours focusing on lab2**
- **Lab 1 …**
  - **Sorry for delay, grades are out**
  - **Review hand graded portion for feedback -> lab2**
- **Exam 1 :** Tue, Mar 10 6:30pm in-lab [sched]

Georgia Tech

Why do we need schedulers?

Do we need schedulers when:

- 1 process in the system?
- 2 processes and 2 CPUs?

Multiple resource consumers

Scarce set of resources shared

Multiplexing

Economic theory

SCHEDULING MOTIVATION

Scheduler is one of the mechanisms in the OS enabling:

- Process isolation
- Resource multiplexing
  - Fairness
  - Resource utilization

SCHEDULING MOTIVATION

Georgia Tech

Why do we need schedulers?

Multiplexing

What do they schedule?

- Processes
- Disk?
- Network?
- Cloud systems?
- AI?

SCHEDULING MOTIVATION

Georgia Tech

Why do we need schedulers?

Multiplexing

What do they schedule?

Processes | **Disk?** | **Network?** | **Cloud systems?** | **AI?**

What is the goal of a scheduler?

(measure of success)

Maximizing the amount of work we care about

SCHEDULING GOALS

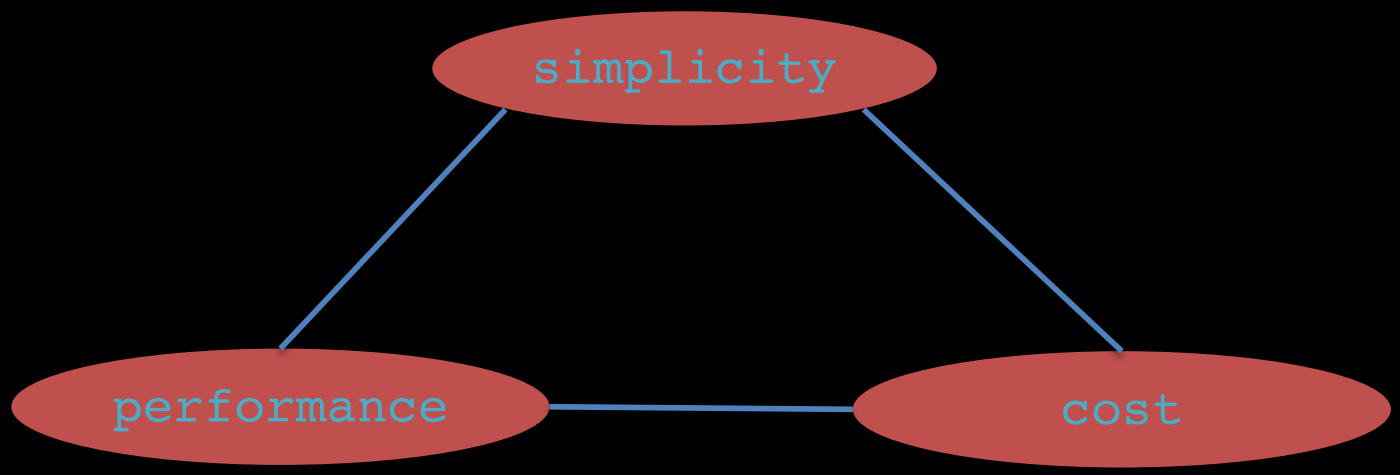
- **Throughput** scheduling (e.g., *disk*)
  - Orders of magnitude performance diff as a f(access pattern)
  - Elevator scheduling
    - *Implicit assumption?*
    - *Support for **declarative** requests? (Coda elevators)*
- Scheduling to minimize **response time**
  - E.g., social media feed
- Scheduling to maximize **latency SLO attainment**
  - E.g. ad serving, LLM inter-token latency, AR/VR
- Scheduling to minimize **cost**
  - Different kinds of cost (e.g., $)

Actuation delay

Optimization Problem Setup

SYSTEMS CONJECTURE

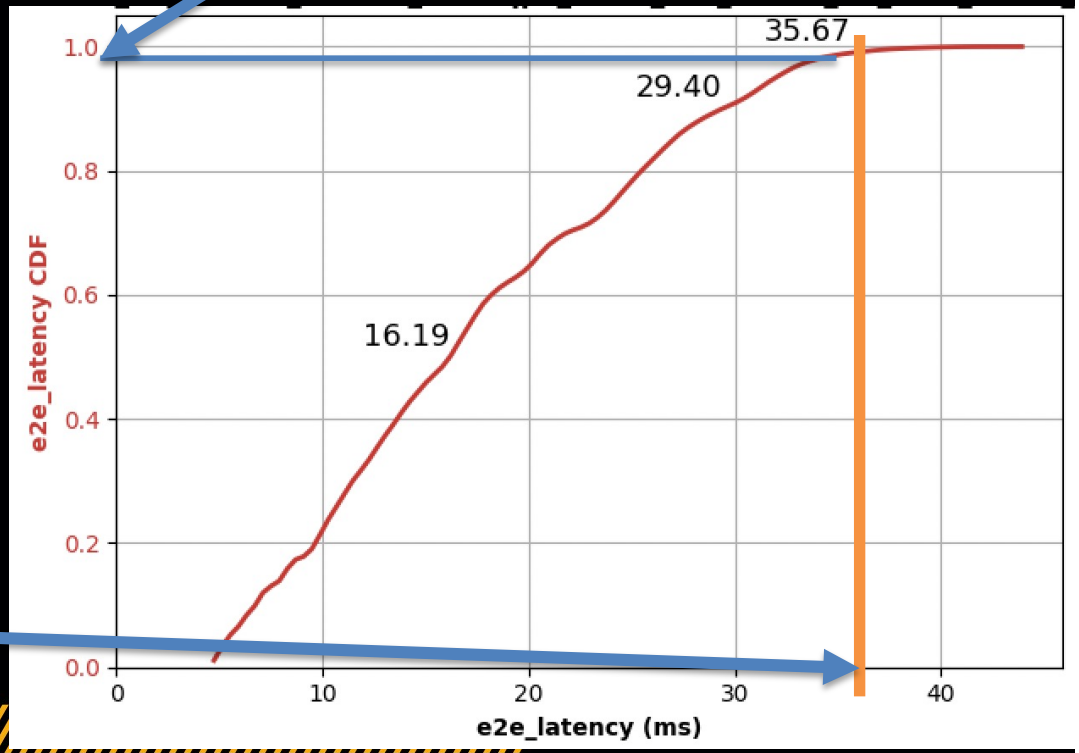
Can’t have all three of



LATENCY DISTRIBUTION FUNCTIONS

- **Probability Distribution Function**
  - **y = P (X = x)**
- **Cumulative Distribution Function**
  - **y = P(X <= x)**
- **What is "tail latency"?**

Tail latency (e.g., 99th %ile)



Latency SLO
e.g. 36ms

SCHEDULER SCENARIO

Imagine two different CPU scheduling scenarios:

One with long running jobs. `fcfs?`

One with very short running jobs. `preemptive?`

Why might a *different* scheduler be better in each scenario?

Examples:

- Delayed scheduling (EuroSys'10)
- Power of two choices
- Spatio-temporal bin packing (e.g., TetriSched EuroSys'16)

`Non work-preserving`

Concept: *fluidity* (sand or water)

spatio-temporal granularity

SCHEDULER DESIGN CONSIDERATIONS

- What should I consider when I build a scheduler?
  - What are the ***costs*** of scheduling?
    - *Costs of interposing on control flow?*
  - What are the ***benefits*** of scheduling?
    - *What do you get in return?*

Quick group breakout session to brainstorm.

SCHEDULING COSTS

- What are the ***costs*** of scheduling?
  - Context Switching
    - Switching Computation
    - Caching costs
  - Scheduler time complexity
  - Predictability? (consider RTOS)
- What are some of the ***benefits***?
  - Better resource utilization
  - More work on important tasks (priority allocation to higher value work)
  - Stronger QOS (Quality of Service) properties

Georgia Tech

Scheduling: Design Space:

Throughput vs latency?

Fairness: time quota, epoch (fairness in heterogeneous context)

QoS: priority? (e.g., nice)

Pre-emptive vs Cooperative?

Global vs Local? (resource state visibility)

Scalability / Performance? (e.g., 1M processes?)

Granularity (timeslice, quantum): 10ms vs 100ms vs *dynamic*?

Constraints: realtime, deadlines (e.g., airplane)

etc: resource starvation, performance consolidation (e.g., cloud)

Georgia Tech

Scheduling: difficult in practice (e.g., distributed operating system)

No perfect/universal solution/policy

Contradicting/heterogeneous goals:

- maximizing throughput vs. minimizing latency
- minimizing response time vs. maximizing scalability
- maximizing fairness vs. maximizing scalability
- Maximizing resource utilization vs. maximizing power consumption

Many potential solutions

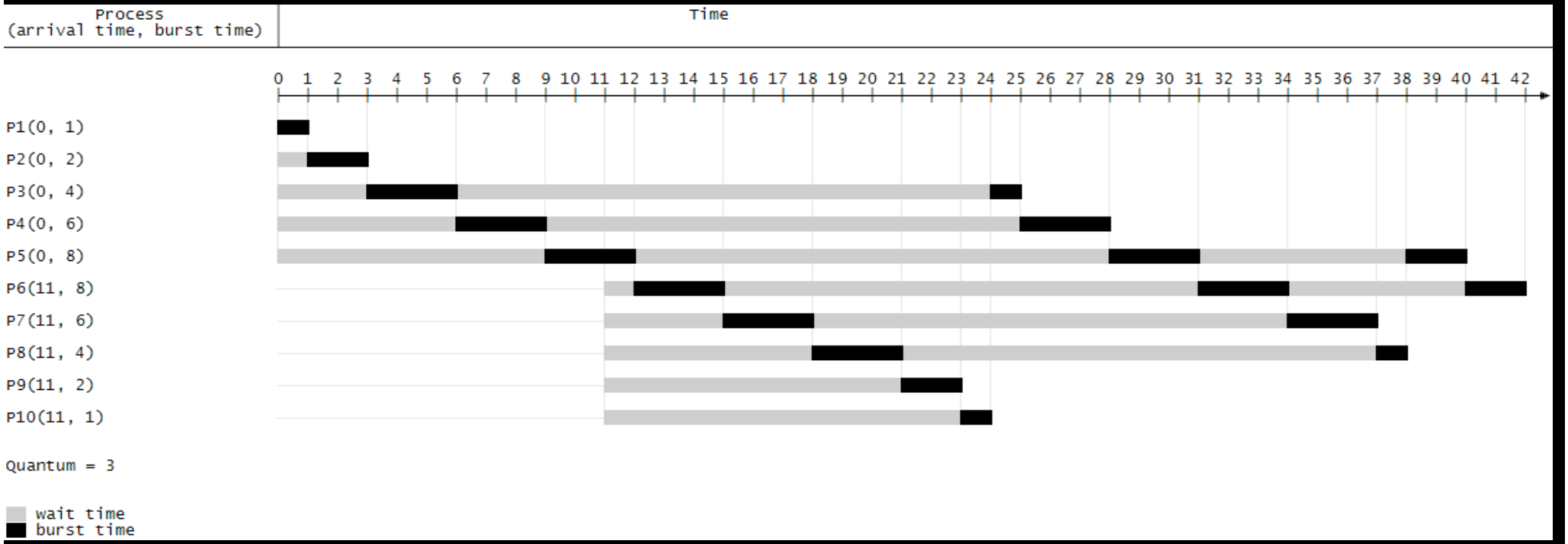
FCFS, Shortest Job First, Priority, Round-Robin, Multi-level Queue, Multi-level Feedback Queue, MinJCT

SCHEDULING

- What priorities does a general-purpose scheduler have?
  - What algorithm should the Linux kernel use?

SCHEDULING: RR PROS AND CONS

Example: round-robin scheduling (as in xv6)



- Advantages?
- Disadvantages?

SCHEDULING: RR PROS AND CONS

- Round Robin advantages

  Simple: assign fixed time unit per process

  Starvation-free (no priority)

- Round Robin disadvantages
  - Doesn’t distinguish urgency/priority of processes
  - Deadline-unaware
  - Doesn’t handle the disparity between throughput and latency

Georgia Tech

Linux Schedule (Modern high-performance)

Enabling/Motivating Observation:

Processes that constantly use full CPU are likely lower priority than those that mostly sleep.

e.g., Network servers need fast turn around time, but spend most of their time sleeping.

Lin

Idea:
Reduce priority from processes that always consume full CPU

Processes that constantly use full CPU are likely lower priority than those that mostly sleep.

e.g. Network servers need fast turn around time, but spend most of their time sleeping.

DRAWING SPACE

Georgia Tech

Linux Scheduler (Modern high-performance)

- WFQ (Weighted Fair Queuing) – Share resources proportionally to process "weight".
  - E.g. If 2 procs run, A with weight 2 and B weight 1, A will get 2x the resources as B
- CFS (Linux's solution) is an implementation of WFQ

- What are the implications of this algorithm if processes sleep?

REAL-TIME SCHEDULING

Divided into two groups:

- **Hard** real-time – Complete critical task within guaranteed time
- **Soft** real-time – Critical receive priority over less critical

NOTES:

Hard real-time *impractical* with secondary storage or virtual memory (why?)

Soft real-time has two requirements:

1) Priority scheduling (*urgency-aware*)

2) low dispatch latency: hard and often not met

e.g., no preemption of system calls)

Georgia Tech

Characterizing processes

CPU-bound vs IO-bound

Interactive processes (e.g., vim, emacs)

Batch processes (e.g., cronjob)

Real-time processes (e.g., audio, video, AR/VR, chatbot interaction):

- presents an interesting throughput vs. latency tradeoff in RT
- care about both qps and real-time success metrics (e.g. TBT)

SCHEDULING

Scheduling policies in Linux

SCHED_OTHER: normal time/schedule sharing (default)

SCHED_IDLE: Very low prioritized processes

SCHED_BATCH: CPU intensive processes

SCHED_FIFO: first in, first out, real time processes

SCHED_RR: round robin real time processes

SCHEDULERS

xv6 scheduler

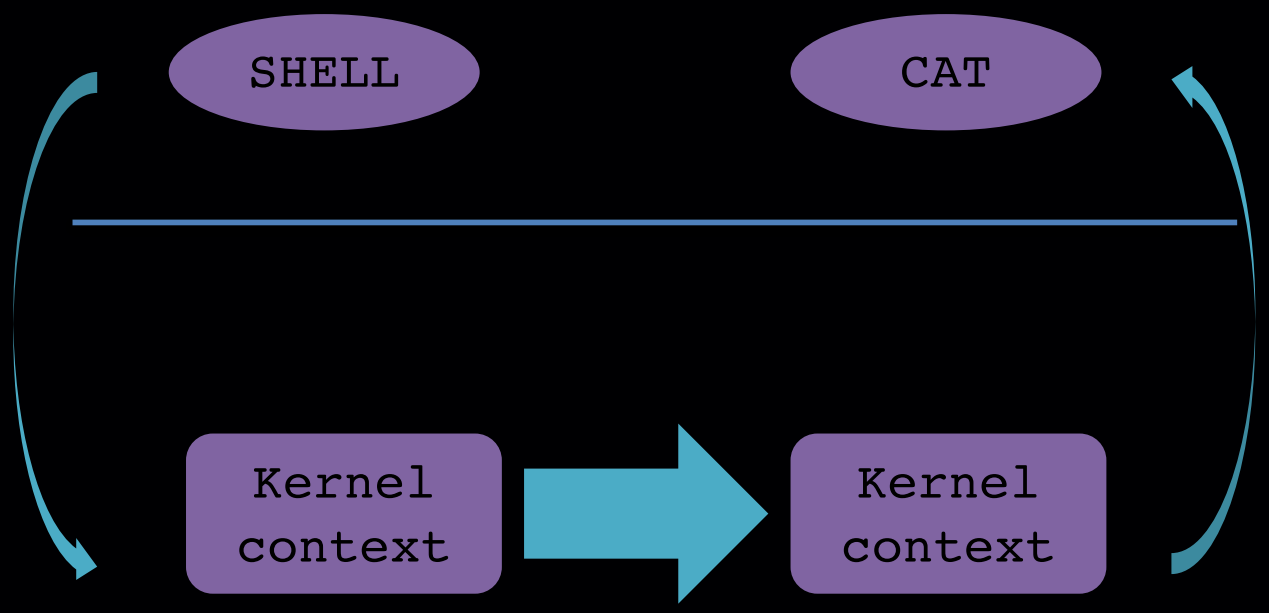
```
// Loop over process table looking for process to run.
acquire(&ptable.lock);
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
  if(p->state != RUNNABLE)
    continue;

  // Switch to chosen process.  It is the process's job
  // to release ptable.lock and then reacquire it
  // before jumping back to us.
  proc = p;
  switchuvm(p);
  p->state = RUNNING;
  swtch(&cpu->scheduler, p->context);
  switchkvm();

  // Process is done running for now.
  // It should have changed its p->state before coming back.
  proc = 0;
}
release(&ptable.lock);
```

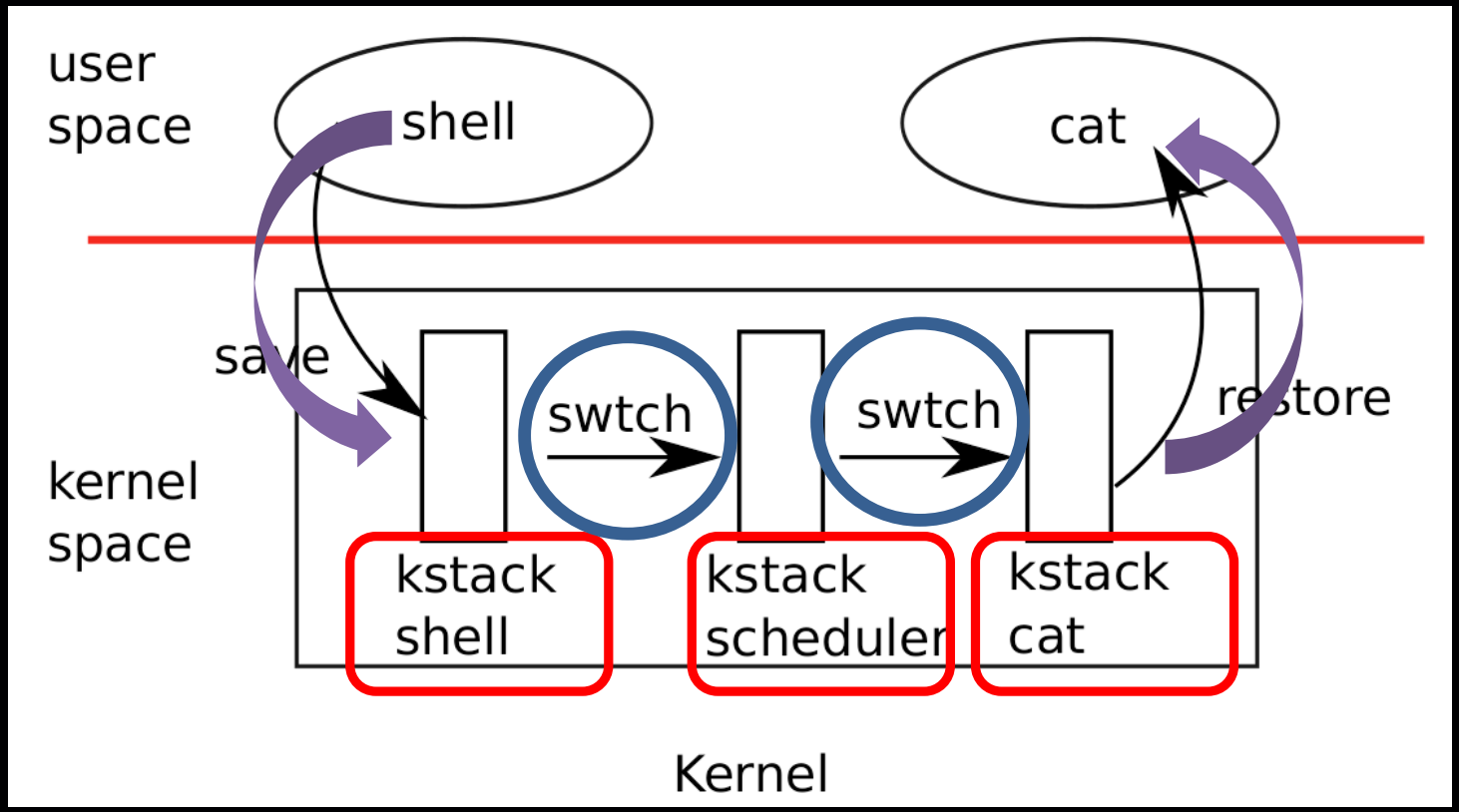
SCHEDULING: CONTEXT SWITCH

Big picture: This is what we *want* to do



Georgia Tech

Big picture: this is what we *have* to do



Georgia Tech

Context Switching: Implementation Challenges

How to switch from one process to another?

- Context switching

How to make context switching transparent?

- Timer interrupts

How to switch among processes running concurrently?

- Locking

How to coordinate processes?

- Sleep on events (e.g., pipe, child exit)

Georgia Tech

Two kinds of context switch

From a process's kernel context to CPU scheduler context

From the scheduler context to a process's kernel context.

xv6 **never** directly switches from *user-space to user-space*

- user-kernel transition (system call or interrupt, incl. timer interrupt)
- context switch to scheduler
- context switch to new process's kernel thread
- trap return

SCHEDULING: CONTEXT SWITCHING

Context switching

- Every xv6 process has its own kernel context:

  kernel stack and register set (incl. %esp, %eip)

- Every CPU has its own scheduler thread
- Switching from one thread to another
  - Save and load CPU registers, including %esp, %eip.

SCHEDULING: SWTCH

swtch

```
void swtch(struct context**, struct context*);
```

Doesn't know about threads just saves and loads sets of registers called contexts

When time to give up CPU, kernel thread calls swtch to save itself and return to scheduler context

Context is a struct context*, stored on the kernel stack

CPU pushed onto stack and saves stack pointer to *old

Copies new to %esp, pops previous registers, and returns

Georgia Tech

yield

At the end of each interrupt, trap can call yield

yield -> sched -> swtch

Switches from proc=>context to cpu=>scheduler

Georgia Tech

swtch: Detailed Look

Loads its arguments off the stack into %eax and %edx before it loses its arguments when it changes %esp

Only callee-save registers saved

- %ebp, %ebx, %esi, %ebp, and %esp
- First four pushed, %esp saved as *old

%eip was saved by the call instruction and is just before %ebp

Moves pointer to new context into %esp

Inverts sequence of steps to load context

Georgia Tech

Scheduling

Process giving up the CPU must

- acquire ptable.lock
- release any other locks (remember deadlocks)
- update its own state (e.g., RUNNABLE, SLEEPING)
- call sched

yield, *sleep*, and *exit* all do these steps

sched and scheduler

Kernel thread always gives up in sched and switches to same location in scheduler

Almost always switches to a process in sched.

Thread switches follow a simple pattern between sched and scheduler

- Coroutines

Exception is forkret when process is first scheduled

Georgia Tech

Scheduler

Loops over process table looking for RUNNABLE processes

Finding one, sets current per-CPU process to proc

Switches page table with switchuvm, marks as RUNNING, and calls swtch

Georgia Tech

References

Arpaci-Dusseau, Remzi H.; Arpaci-Dusseau, Andrea C. (2014). "Multi-level Feedback Queue" (PDF). Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books

Avi Silberschatz, Peter Baer Galvin, and Greg Gagne. 1999. *Applied Operating System Concepts*. John Wiley & Sons, Inc., New York, NY, USA.

Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, et al.. *The Linux Scheduler: a Decade of Wasted Cores.* EuroSys 2016, Apr 2016, London, United Kingdom. Proceedings of the Eleventh European Conference on Computer Systems. . .

https://www.ibm.com/developerworks/linux/library/l-completely-fair-scheduler/